

Green Fleet Optimizer — Understanding Our Project

This document explains the whole project in plain language: what we built, why each part exists, how the parts connect, what tools we used and why, and what our results mean. Every technical word is explained the first time it appears (highlighted **like this**), and there is a full glossary at the end. Read it once end-to-end before the presentation; then use the glossary and the "how to answer" section for revision.

PART 1

The big picture, in five minutes

What problem are we solving?

Ships carry most of the world's goods. India moves about 95% of its trade by volume through its ports. Ships burn a heavy, dirty fuel, which creates two problems at once: it is expensive, and it produces greenhouse gases that warm the planet.

Governments now want ships to switch to cleaner fuels — **LNG** (liquefied natural gas, natural gas cooled to a liquid), **methanol** (a liquid alcohol fuel), **hydrogen**, and **ammonia** (a nitrogen-hydrogen compound that burns without releasing carbon dioxide) — or to plug into the electricity grid while docked (called **shore power**).

India has its own targets. The **Harit Sagar** guidelines ("Green Ocean" in Hindi) say ports must cut pollution per tonne of cargo by 30% by 2030 and 70% by 2047. The **Green Tug Transition Programme (GTTP)** is replacing diesel harbour tugboats with electric or clean-fuel ones.

But here is the hard part. A port or shipping company must answer many questions *at the same time*: which ships to use, how fast to sail them, which fuel each should burn, whether cargo still arrives on time, whether the fuel is even available at that port, and whether the pollution rules are met. Every answer changes the others. With just 20 ships, 4 fuels and 5 speed settings, there are more combinations than anyone could ever check by hand.

Why this matters to a judge: The problem is real, Indian, and current. There is a live government programme (GTTP), a live target (Harit Sagar), and a live pilot (our problem-statement sponsor signed an agreement with Mormugao Port in July 2026). We are not inventing a need.

What does our software do?

It has two "engines":

1. **A fuel predictor.** Given a ship, a speed and a load, it predicts how much energy the ship will need. It learned this from 26,000 real ships.

2. **A fleet planner (the optimizer).** It searches through millions of possible plans — which ships, which fuels, which speeds — and returns the best trade-offs between cost, fuel and pollution, while obeying every rule.

Between the two sits an **emissions engine** that counts pollution honestly (more on this below — it is our biggest advantage), and an **India layer** that knows which fuels are available at which Indian ports in which years.

Why "quantum-inspired"?

The problem statement requires it. But be very clear about what it means:

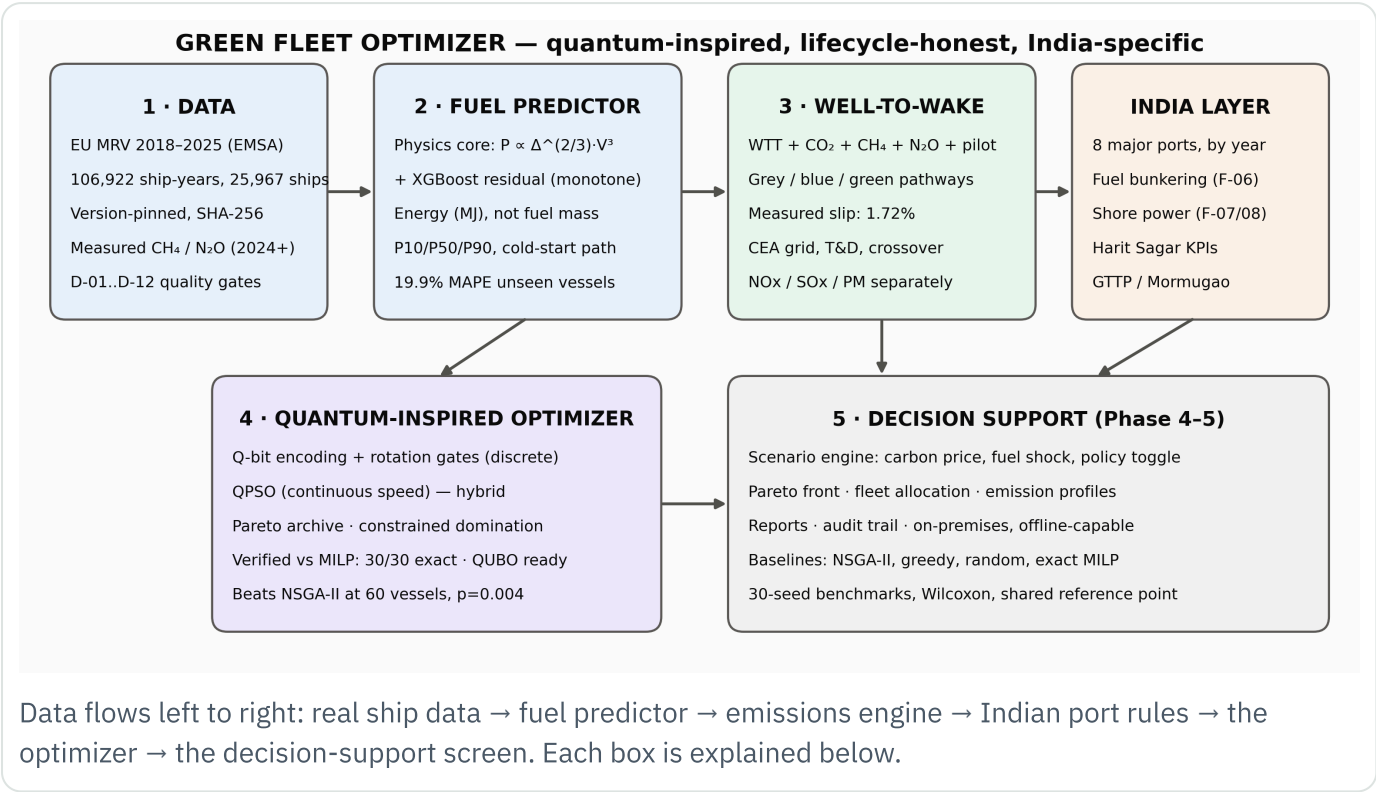
Quantum-inspired means a normal computer program that *borrowes ideas* from quantum physics to search smarter. It is **not** a quantum computer. It runs on an ordinary laptop. We never claim a "quantum speedup". If a judge asks "is this real quantum computing?", the answer is a clear **no**, followed by what it actually is.

The idea it borrows: in a normal search, each choice is either yes or no. In our search, each choice starts as a *probability* — "60% chance this ship uses methanol". The search keeps many possibilities open at once (like a quantum bit that is in a mix of states), and slowly nudges the probabilities toward choices that turn out to be good. This helps it explore widely early on and avoid getting stuck on a "good but not best" answer.

Why is ours better than existing tools?

Companies like ZeroNorth and Wärtsilä sell software that helps **one ship on one voyage** — best route, best speed. Good tools. But they do not decide *which ships to buy* or *which fuels to commit to* — and that is the decision a port under GTTP actually faces. Also, they count only the pollution that comes out of the ship's funnel. We count the whole life of the fuel, and that changes the answer completely (Part 3 explains how).

The flow — how the project works, step by step



1 Get real data

We download eight years (2018–2025) of real ship fuel records from the European Union's official database. Every ship over 5,000 tonnes calling at a European port must report its fuel use and CO₂ by law, and an independent auditor checks it. This is 106,922 ship-years from 25,967 real, named ships. **Nothing is made up.**

2 Clean the data

Real data is messy. We found ships reporting 603 million hours at sea (68,000 years — a typo), pollution levels physically impossible for any fuel, and the literal text "Division by zero!" in official EU spreadsheets. Twelve automatic checks catch these. We never silently delete or guess — every problem row is flagged and counted.

3 Predict fuel use

We combine a physics law (fuel rises roughly with the *cube* of speed) with machine learning (which learns each ship's individual efficiency from its design). Result: on 6,263 ships the model had never seen, it predicts energy within 19.9% on average.

4 Count emissions honestly

We add up pollution from making the fuel, transporting it, and burning it — including two gases most tools ignore (methane and nitrous oxide). This is called "well-to-wake". It flips the verdict on three fuels.

5 Apply India's rules

Which fuels can be bought at Mumbai, Mormugao, Cochin in 2030? Is shore power installed? How dirty is India's electricity? Are we on track for Harit Sagar? All of this is data the optimizer must obey.

6 Optimize the fleet

The quantum-inspired search explores the millions of combinations and returns not one answer but a *set* of best trade-offs — "cheapest plan", "cleanest plan", and every sensible option between.

7 Prove it works

We compare against standard methods on the same problem with the same budget, 30 times each with different random starts, and use a statistical test to check the difference is real. On tiny problems we compute the exact best answer by brute force and check we find it.

8 Show it (next phase)

A dashboard where a port officer can enter their fleet, try "what if" scenarios (fuel price shock, carbon tax, hydrogen becomes available in 2030), see the trade-offs on a chart, and download a report.

The data: where it comes from and why it matters

EU MRV = European Union Monitoring, Reporting and Verification. A law (since 2018) that makes every large ship visiting Europe report its yearly fuel consumption and CO₂. The reports are checked by an accredited verifier, then published by **EMSA** (the European Maritime Safety Agency). We downloaded them from EMSA's public system called **THETIS-MRV**.

Why real data instead of made-up data?

Because judges will ask "where does your data come from?", and "we simulated it" is the weakest possible answer. Ours is: *"106,922 verified regulatory filings from 25,967 real ships, 2018 to 2025, downloaded from EMSA's public API."* No competitor team is likely to have this.

What is in it?

For each ship, each year: total fuel burned, total CO₂, hours at sea, fuel per nautical mile, fuel per tonne of cargo moved, the ship type, and a "design efficiency" number from the ship's certificate. From 2024 onwards it also includes **measured methane and nitrous oxide** — which turned out to be extremely valuable (see Component 3).

Two honest limits of this data

- **No tugboats.** The law only covers ships over 5,000 **GT** (gross tonnage, a measure of a ship's size). A harbour tug is 200–500 GT. So no tug appears in our training data. For tugs we use physics and published tug specifications, and we *label* those numbers as physics-derived, not data-derived. Say this before a judge finds it.
- **Yearly totals, not voyage-by-voyage.** We know a ship's total fuel for a year, not its fuel on each trip. This leads to the "circularity trap" below.

Provenance: why we record file versions

EMSA re-publishes a year's file every time a company corrects a filing (the 2024 file is at version 242). We record the exact version and a **SHA-256** (a digital fingerprint of the file) so anyone can reproduce our exact result later. Judges who work in research respect this.

The circularity trap — our most important early discovery. We wanted to learn from the data how fuel changes with speed. But in MRV data, "speed" and "fuel per mile" are both *calculated from the same column*. Fitting one against the other is partly fitting a number against itself. We proved this mathematically (the two are linked by an exact identity, error 5×10^{-16}) and confirmed it experimentally: the fitted speed exponent came out near 1 instead of the physical value 3. A team that missed this would build an optimizer that underestimates the benefit of slowing down by roughly 3× — and slowing down is the single biggest fuel-saving lever there is. We fixed it by taking the speed law from physics (see Component 2) instead of trying to learn it from data that cannot teach it.

The fuel predictor: physics plus machine learning

The physics part (the "shape")

Naval architects have known for a century that the power a ship needs rises with the **cube of its speed**. Slow down from 14 to 12 knots and the engine needs about 37% less power per hour. Because the trip takes longer, fuel per trip drops by about 27%. This is well established, so we *impose* it in the model rather than trying to re-learn it from data that cannot teach it.

Knot = one nautical mile per hour, about 1.85 km/h. **Nautical mile (nmi)** = 1,852 metres. Ships use these units, so we do too.

We also separate two kinds of power: **propulsion** (moving the ship, scales with speed³) and **hotel load** (lights, air-conditioning, refrigeration, pumps — roughly constant whatever the speed). This separation matters: a cruise ship's hotel load is huge, so slowing it down saves almost nothing per mile, while a bulk carrier saves a lot. Our model captures that; a simple "cube law" tool cannot.

The machine-learning part (the "level")

Physics tells us the *shape* of the curve; it does not tell us how efficient one particular ship is. That is what the machine learning learns — from the ship's type, its design-efficiency certificate, whether it is ice-strengthened, and so on.

XGBoost = a widely used machine-learning method that builds many small decision trees and adds them up. It is fast, accurate on tabular data, and — importantly for us — can be told "this input must only ever push the answer in one direction", which we use to keep the physics honest. That is called a **monotone constraint**.

Why we predict energy, not fuel

Methanol carries only about half the energy per kilogram that diesel does. If the model predicted "tonnes of fuel", it would be wrong the moment you changed fuel. So it predicts **energy in megajoules (MJ)**, and a separate table converts that to tonnes for each fuel using each fuel's energy content (called **LHV**, lower heating value — the energy released per kilogram burned).

How we tested it honestly

Leakage = the most common silent mistake in this kind of project. If the same ship appears in both the training data and the test data, the model just "remembers" it and the accuracy looks fantastic but is fake. We split by *whole ships* — 25% of ships are held back entirely — and we check by code that no ship appears on both sides. We also test on *future years* the model never saw.

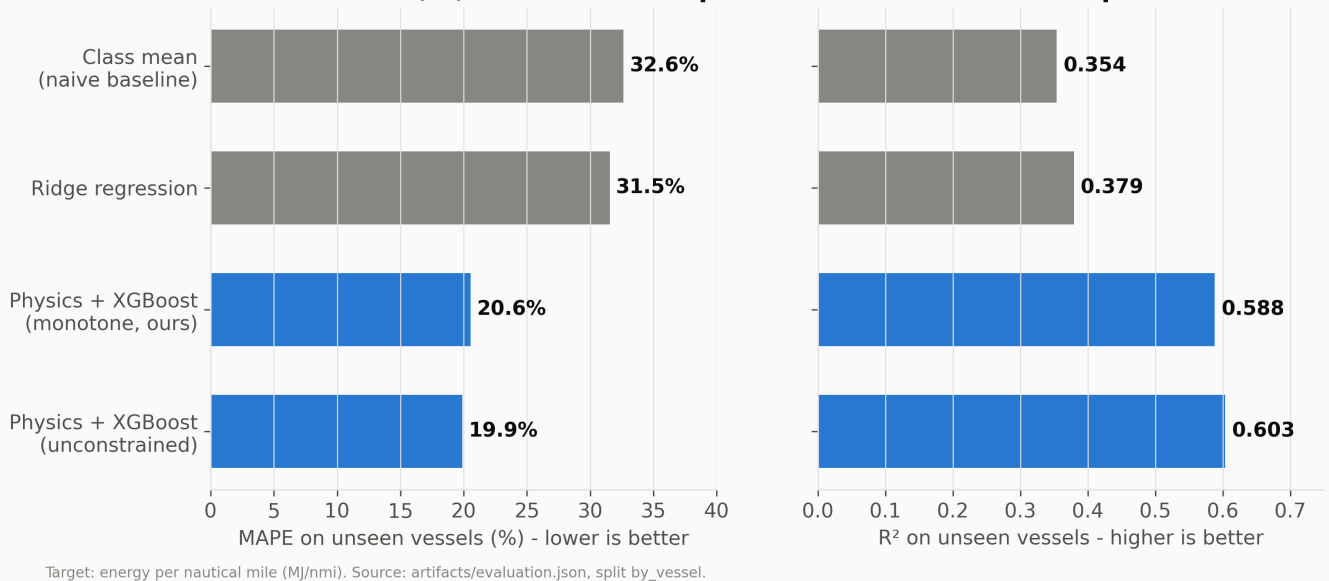
MAPE = Mean Absolute Percentage Error. If the ship really used 100 units and we predicted 120, that is a 20% error. MAPE is the average of those percentages. Lower is better. **R²** ("R

squared") = how much of the variation between ships the model explains, from 0 (nothing) to 1 (everything).

Leakage-safe: 25% of ships fully held out, zero IMO overlap, 100,528 ship-years

Prediction error, 6,263 held-out ships

Variance explained



On 6,263 ships never seen in training: our physics + XGBoost model reaches 19.9% MAPE, R^2 0.603. A naive guess (the average for that ship type) gets 32.6%.

"The model knows what it doesn't know" — the cold-start path

Some ships have no design certificate (most offshore vessels, and all tugs). Instead of guessing confidently, the model flags these as **cold-start** (a ship we have almost nothing on), gives a physics-based estimate, and *widens the uncertainty band*. We report warm and cold ships separately: warm 19.3% error, cold 41.8% — and the cold band covers the truth 84% of the time, as it should.

P10 / P50 / P90 = instead of one number, we give three: a low estimate (10% chance the truth is below it), a middle estimate, and a high one (10% chance the truth is above). The band between P10 and P90 should contain the truth about 80% of the time. Ours contains it 80.0% of the time on unseen ships — that means the uncertainty is *honest*, and the planner can safely use the conservative end.

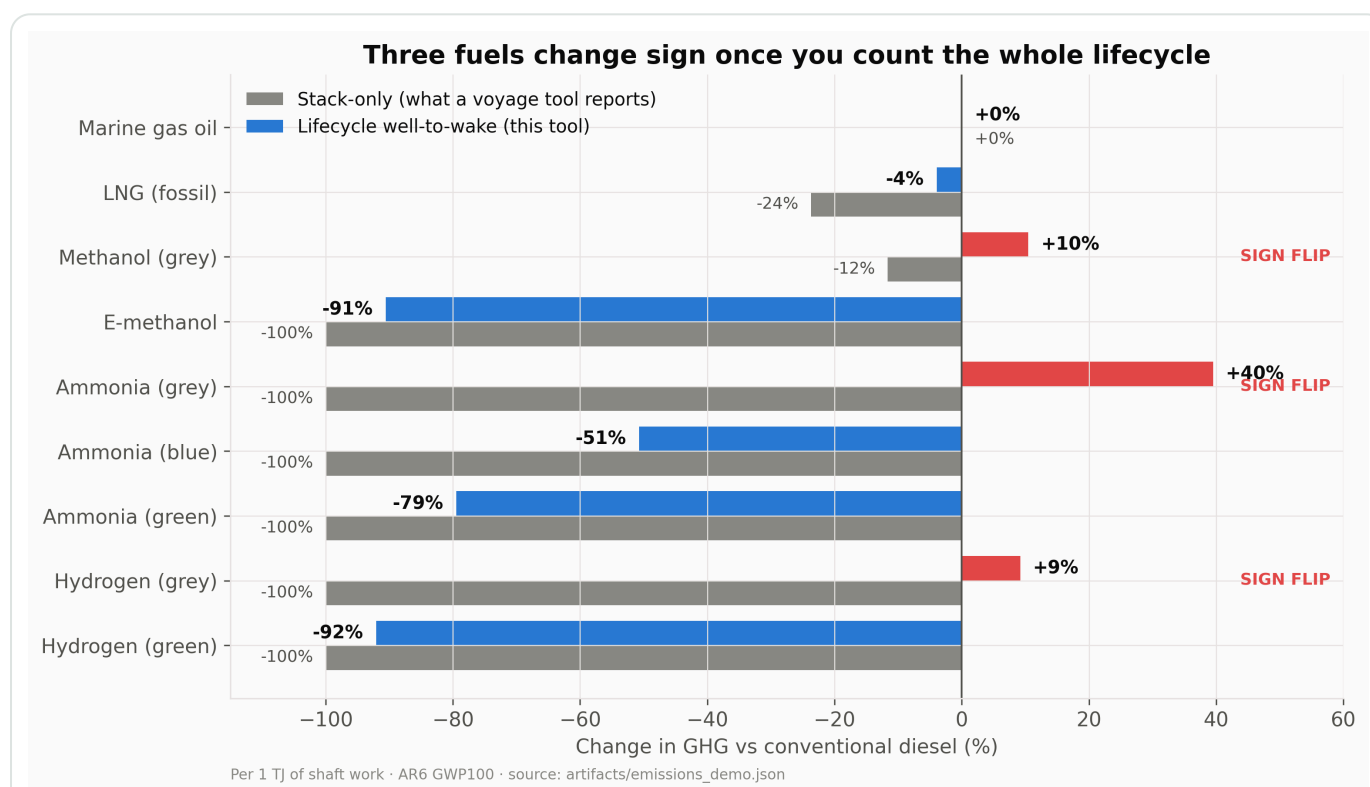
Why judges will care: "How accurate is it?" has a defensible answer with a leakage-safe split. "What about ships with no data?" has a built-in answer (cold-start). "Does slowing down really save fuel?" — the physics is imposed, so the answer is always right by construction.

The emissions engine: counting the whole life of the fuel

Well-to-wake = the full lifecycle of a marine fuel. "**Well-to-tank**" is everything before the ship: extracting or making the fuel, refining it, shipping it to the port. "**Tank-to-wake**" is burning it on board. (The "wake" is the trail behind a ship.) Most tools count only tank-to-wake — what comes out of the funnel.

Why this is our biggest advantage

Ammonia contains no carbon, so burning it releases no CO₂. A funnel-only tool therefore scores ammonia as a perfect, zero-emission fuel. But *making* ammonia from natural gas (the usual way, called **grey** ammonia) releases so much CO₂ upstream that grey ammonia is **40% worse than diesel** over its whole life. Made with renewable electricity instead (**green** ammonia), it is 79% better. Same molecule, opposite verdict — the *pathway* is everything.



Grey bars: what a funnel-only tool reports. Blue/red bars: the truth. Three fuels flip from "improvement" to "worse than diesel". A port buying on the grey bars would increase pollution while reporting it had reached zero.

The two gases most tools forget

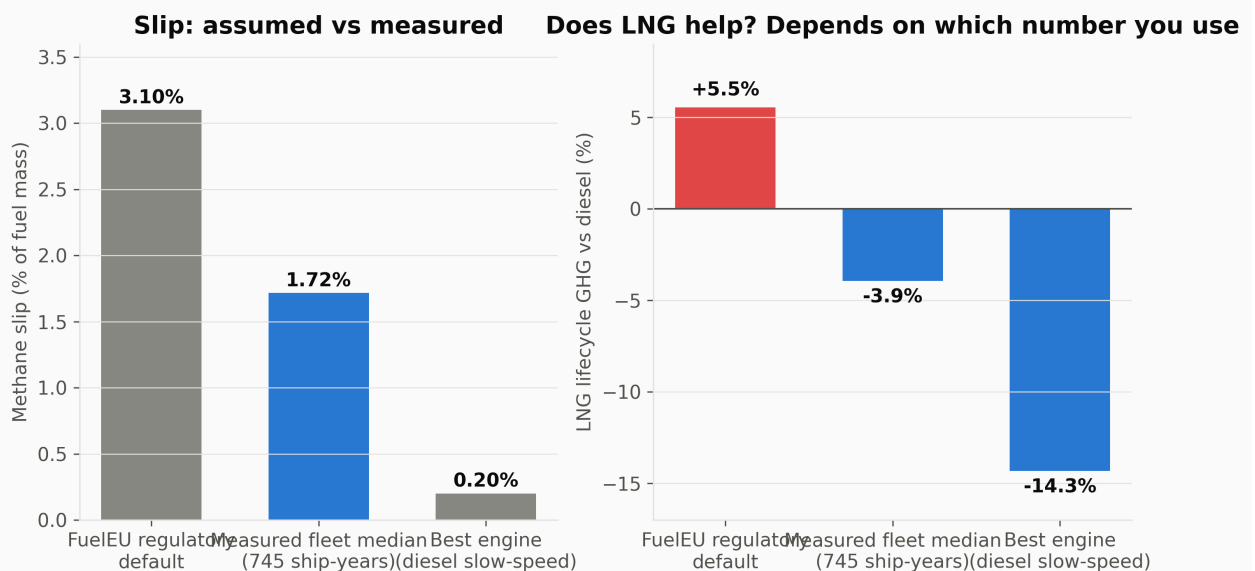
Methane slip = LNG is mostly methane. Some of it passes through the engine unburnt and escapes. Methane traps about 28–30 times more heat than CO₂ over a century, so even 1–3% escaping erases most of LNG's advantage. **N₂O (nitrous oxide)** = a gas formed when ammonia burns; it traps about 265–273 times more heat than CO₂.

GWP = Global Warming Potential. A multiplier that converts a gas into "CO₂-equivalent" (written **CO₂e**) so different gases can be added up. It depends on the time horizon (we use 100 years, "GWP100") and on which scientific report you take it from (IPCC AR5 or AR6). We state which one we use every time — and we discovered, by solving equations on 28,125 real filings, that the EU uses the AR5 numbers (CH₄ = 28, N₂O = 265), exact to 14 decimal places.

We measured methane slip instead of assuming it

The official European default for methane slip is 3.1%. We had real measured methane from 745 LNG carrier ship-years in the 2024–25 data. The real median is **1.72%** — about half. And the two numbers disagree on the sign: with the default, LNG is 5.5% *worse* than diesel; with the measured value it is 3.9% *better*. Being able to say "our methane number is measured from 745 real ships, not a default" is a direct answer to "isn't LNG a clean fuel?"

The regulatory default and the measured fleet disagree about LNG's sign



Left: the regulatory default (3.1%) versus what the real fleet actually does (1.72%). Right: what that does to LNG's verdict versus diesel.

Pilot fuel

Engines that burn methanol or ammonia need a small squirt of diesel to ignite each stroke (called **pilot fuel**, 4–7% of the energy). We count it. Without it, "green ammonia" looks cleaner than it is.

A bug we caught that would have sunk the pitch

Shaft energy vs input energy. Our first version showed electric tugs as 146% *worse* than diesel. That was wrong. A marine diesel turns only about 45% of its fuel energy into useful work at the propeller; an electric motor turns about 92% of its electricity into work. Comparing on "energy in" instead of "work out" punishes the battery by a factor of two. We now compare everything on **shaft energy** (work delivered at the propeller shaft). The Green Tug Transition Programme is *the* flagship Indian programme this project targets; a tool that

told ports not to electrify because of a units mistake would have been fatal. We show this bug and its fix openly — it demonstrates rigour.

Local pollution is not the same as climate pollution

CO₂ warms the planet but does not hurt your lungs. **NOx** (nitrogen oxides), **SOx** (sulphur oxides) and **PM** (particulate matter, soot) do not warm the planet much but cause asthma and heart disease in port cities. We track them *separately* and never add them into the CO₂e total (that would be like adding kilograms to kilometres). This matters for the electrification story below.

The India layer: rules the optimizer must obey

A clean fuel is useless if you cannot buy it at the port. We built a data file describing eight major Indian ports (Mormugao, Deendayal, Paradip, V.O. Chidambaranar, JNPA, Visakhapatnam, Cochin, Mumbai): which fuels can be bunkered there, in which year, whether shore power is installed, and how dirty the electricity is.

Bunkering = refuelling a ship. **Shore power** (also "cold ironing" or OPS, onshore power supply) = plugging a docked ship into the grid so it can switch off its engines. **CEA** = Central Electricity Authority, which publishes how much CO₂ India's grid emits per unit of electricity. **T&D losses** = electricity lost in transmission and distribution wires before it reaches the ship (about 17% in India), which we add on top.

Three rules we enforce

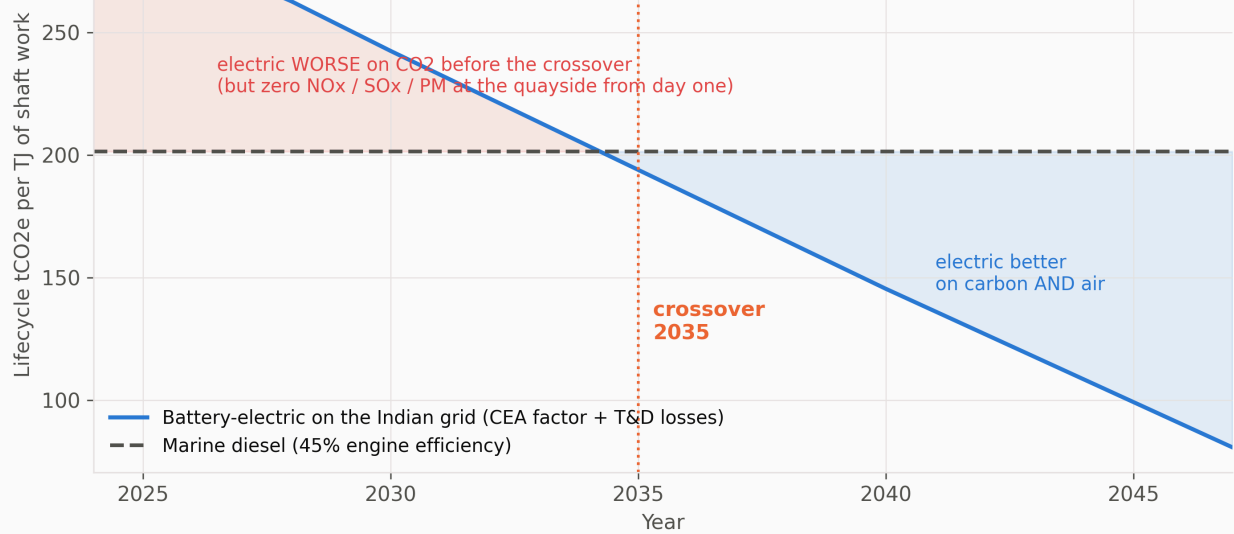
- **A fuel must be available at every port on the route.** One gap strands the ship. And "planned" does *not* count — planning a fleet around infrastructure that is only announced is exactly the mistake this prevents.
- **Shore power needs both the port and the ship to be equipped.** Either missing means the ship runs its own engines at the dock.
- **Shore power is only as clean as the grid.** India's grid is coal-heavy, so this is a real number, not zero — and it improves every year.

The honest electrification answer

Against a 45%-efficient marine diesel, India's current grid is not clean enough: a battery-electric tug emits **20% more CO₂** in 2030 and only crosses over to better in **2035**. That sounds like bad news for GTTP — until you add the other half:

Each electrified tug removes 35.6 tonnes of NO_x, 1.25 tonnes of SO_x and 187 kg of soot from the port city every year, from day one. So the recommendation is: electrify now for public health; the carbon benefit arrives around 2035 as the grid decarbonises. This is specific, uncomfortable, and obviously not sales copy — which is why it makes a government audience trust everything else we say.

Electrification in India: the carbon benefit arrives with the grid, not the vessel



Grid glide path consistent with the 500 GW non-fossil target; projected years are marked provisional in config/ports.yaml

Blue line: an electric tug on India's grid. Dashed line: diesel. They cross in 2035. The red region is where electric is worse on carbon (but still zero local pollution).

Tank size and range — why hydrogen is a tug fuel

Hydrogen carries very little energy per litre once you include the heavy insulated tank it needs. If you keep a ship's diesel tank and switch it to hydrogen, its range drops to **7%** of what it was. That is why hydrogen suits harbour tugs and ferries (short trips) but not long-haul ships. The optimizer knows this and will refuse to assign hydrogen to a route it cannot complete. Low-density fuels also eat cargo space: a purpose-built hydrogen tank for a 5,000-mile route displaces 4,842 tonnes of cargo.

Harit Sagar tracking

The target is pollution *per tonne of cargo*, -30% by 2030 and -70% by 2047 from a baseline. Our tracker draws the required glide path between those years and tells a port whether any plan is on or off track. In our example, a port switching 35% of its fuel to e-methanol by 2030 is on track.

The optimizer: the quantum-inspired part

The problem, stated simply

For every candidate ship, decide four things: **use it or not**, **which route**, **which fuel**, and **what speed**. Choose so that fuel, pollution and cost are all as low as possible, while every rule is obeyed: all cargo delivered, schedules met, pollution cap respected, fuel available, range sufficient, ship fits the port.

The first three decisions are "pick one from a list" (called **discrete**). Speed is a smooth number, any value between the engine's minimum and maximum (called **continuous**). Handling both together is the tricky part, and we use two quantum-inspired methods, one for each.

Q-bits for the discrete choices

Q-bit = our representation of one yes/no decision. Instead of storing "yes" or "no", it stores a *probability* of yes (technically a pair of numbers called amplitudes whose squares add to 1, exactly like a quantum bit). "Use this ship?" starts at 50/50. **Observation** = rolling the dice to turn every Q-bit into a concrete yes/no, giving one complete plan we can score. **Rotation gate** = after scoring, we nudge each Q-bit's probability toward the value it has in the best plan found so far — but only where the current plan disagreed with a better one. Small nudges, many rounds. This is the "quantum update mechanism" the problem statement asks for.

Choosing a route from, say, 6 options takes 3 Q-bits (binary code). We decode the 3 bits "modulo 6" so that *every* possible bit pattern maps to a real route — no wasted, invalid answers. For 60 ships, 6 routes and 4 fuels that is 360 Q-bits.

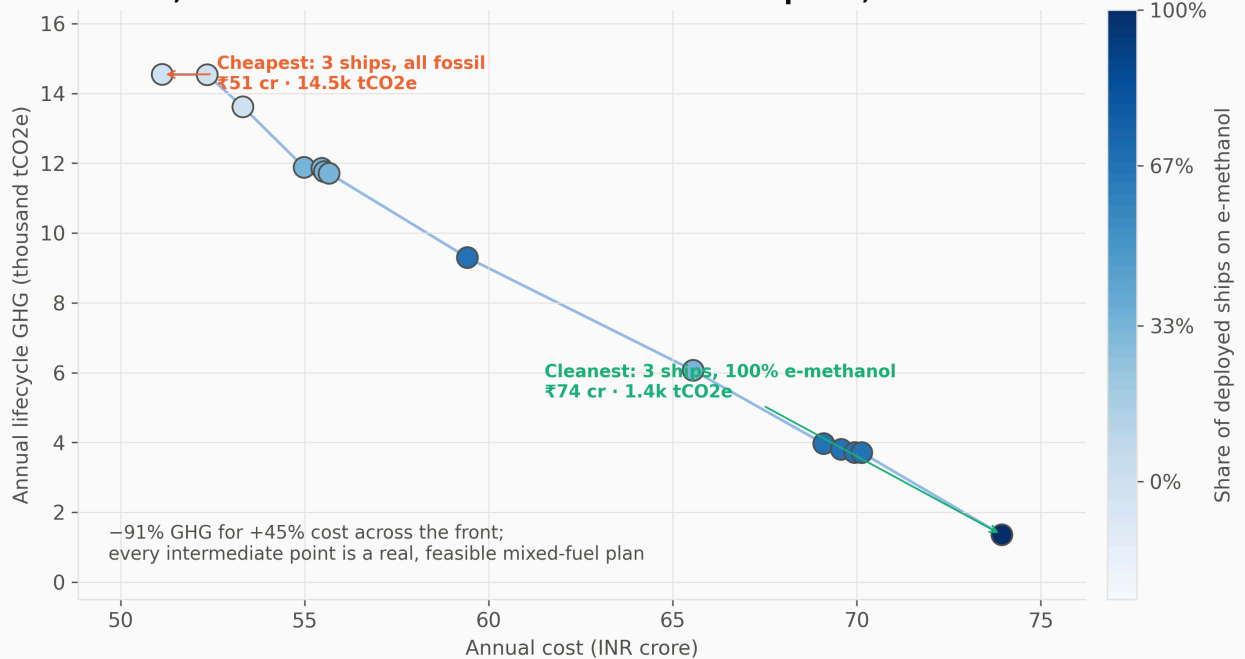
QPSO for the continuous speeds

QPSO = Quantum-behaved Particle Swarm Optimization. Imagine a swarm of bees, each holding a guess at all the ships' speeds. In ordinary "particle swarm" methods each bee flies with a velocity toward the best spots. In QPSO there is no velocity: each bee's next position is *sampled from a probability cloud* around the best spots — derived from the maths of a quantum particle in a potential well. A bee can occasionally "tunnel" far away, which stops the swarm getting stuck.

Why not one best answer? The Pareto front

Pareto front = the set of plans where you cannot improve one thing without worsening another. The cheapest plan is dirtier; the cleanest plan is dearer; between them are plans that trade a bit of one for a bit of the other. We return the whole set and let the decision-maker choose. A single "best" number would hide the trade-off they are actually being asked to make.

A Pareto front, not one answer: 14 non-dominated fleet plans, coastal scenario 2047



12 candidate vessels · 3 Indian coastal routes · year 2047 · MGO/HFO/LNG + e-methanol · every point meets all cargo demand and every constraint

Fourteen feasible plans for a 12-ship coastal fleet in 2047. Bottom-left is impossible (that would be cheap AND clean). The front runs from "all fossil, ₹51 crore, 14,500 t CO₂e" to "100% e-methanol, ₹74 crore, 1,400 t" — a 91% cut for 45% more cost, with real mixed-fuel plans between.

How we handle rules (constraints)

A plan that drops cargo is not a "cheaper plan" — it is not a plan. So a feasible plan always beats an infeasible one, no matter the score. If the demand is genuinely impossible for the fleet, the tool *refuses* and says by how much: "demand 1,000,000 t exceeds the fleet's best case 700,000 t, short by 300,000 t". It never silently returns a broken answer.

A design fix we made, and what we learned

Our first version had every bee follow the *same* leader each round, which pulled the whole swarm into one corner and gave a thin Pareto front. Giving each bee its own leader (chosen from a sparse region of the front) tripled the number of plans found. Second, the rotation-gate step size we had tuned on a small problem was far too slow for a 360-Q-bit problem; we made it scale with problem size, and recorded the failed intermediate attempt too. Both are in the findings documents.

Proving it works: benchmarking honestly

The problem statement's sponsor is a quantum company. They will probe any hype. So we compare our optimizer against standard methods under strictly fair conditions and **publish the cases where we don't win**.

The three levels of proof

1 Exact answer on a tiny problem

For a 2-ship problem we compute the true best answer by **MILP** (mixed-integer linear programming — an exact mathematical solver that proves it has found the optimum). Our optimizer must find that same answer. It does: **30 out of 30 random starts, 0.0000% gap**. A method that fails a trivial case cannot be trusted on a big one.

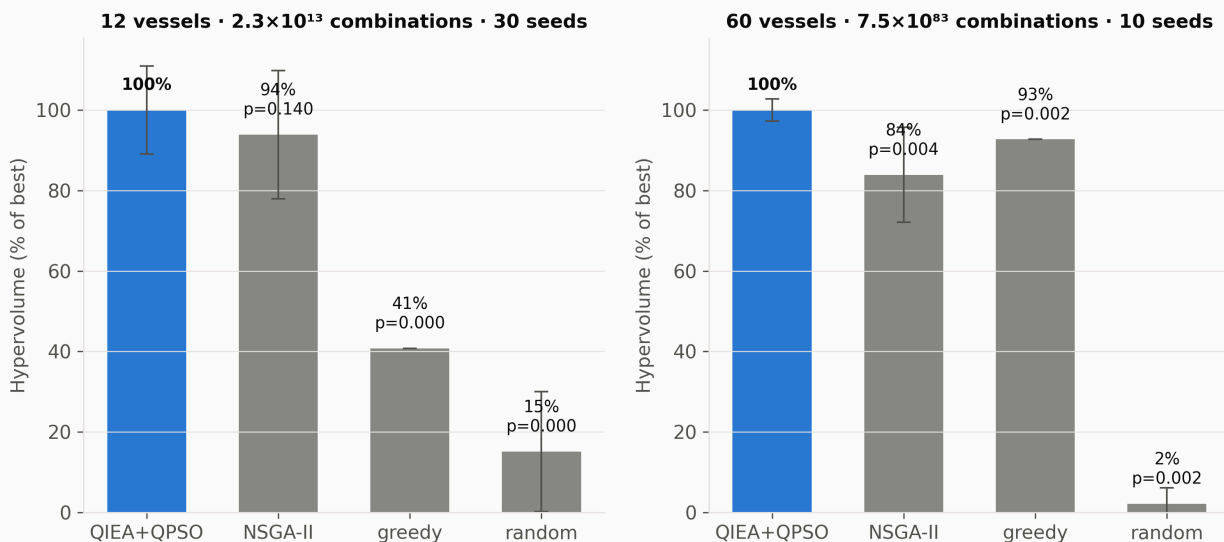
2 Fair comparison on realistic problems

Against **NSGA-II** (the standard genetic algorithm for multi-objective problems), a greedy heuristic (what a planner would do in a spreadsheet), and random search. Every method gets the *same number of evaluations* (6,000). Every method is run 30 times with different random seeds. We measure **hypervolume** (how much of the trade-off space the front covers — bigger is better) against one shared reference point, and use the **Wilcoxon test** (a statistical test for "is this difference real or luck?") paired by seed.

3 Scaling

Does it still work at 200 ships? Yes — 200 vessels, 10^{279} combinations, a feasible plan in 54 seconds, every run.

Quantum-inspired optimizer vs conventional methods, equal evaluation budget (6,000)



Shared reference point · Wilcoxon signed-rank paired by seed · large instance: QIEA better than every baseline, $p \leq 0.004$

Left, 12 ships: our method and NSGA-II are statistically tied ($p = 0.14$ — say "tie", not "win"). Right, 60 ships: ours beats every method with $p \leq 0.004$. The advantage appears when the problem is large — exactly what the theory predicts.

p-value = the probability that a difference this big would appear by luck if there were really no difference. Below 0.05 is conventionally "significant". $p = 0.004$ means about a 1-in-250 chance of luck. $p = 0.14$ means we cannot claim a real difference — so we call it a tie.

The honest pattern across everything we tested: on small or smooth problems (tuning XGBoost settings, choosing 10 features, a 12-ship fleet) the quantum-inspired method *ties* conventional methods. On large, rugged problems (a 4,096-state knapsack, a 60-ship fleet, a multimodal maths benchmark) it *wins clearly*. That is a narrow, defensible claim. "It always wins" would be false and a quantum-expert judge would know it.

"Quantum-ready" — what QUBO means

QUBO = Quadratic Unconstrained Binary Optimization. A standard mathematical form (minimise $x \cdot Q \cdot x$ over 0/1 variables) that real quantum annealers — like D-Wave machines — consume directly. We wrote the fleet problem in this form and verified that its lowest-energy state *exactly* matches the exact MILP answer. That means the same problem could run on quantum hardware when India's National Quantum Mission makes it available — without reformulation. We have **not** run it on a quantum device, and we say so.

The tools we use, and why each one

TOOL	WHAT IT IS	WHY WE CHOSE IT
Python 3.10	The programming language	The standard for data science and optimization; every library below is Python-first
NumPy	Fast numerical arrays	Our quantum-inspired algorithms are written directly in NumPy so we control every line — no black box
pandas	Tables and data cleaning	Reading 106,922 rows across 8 Excel files with changing column names
scikit-learn	Machine-learning toolkit	Leakage-safe data splitting (grouped by ship) and baseline models
XGBoost	Gradient-boosted trees	Best accuracy on tabular data, and supports monotone constraints that keep physics honest
pymoo	Multi-objective optimization library	Gives us NSGA-II, the standard baseline we must beat, implemented by its own authors — no accusation of a weak baseline
OR-Tools CP-SAT	Google's exact solver	Proves the true optimum on small problems so we can verify our method (Q-08 in the test register)
SciPy	Scientific statistics	The Wilcoxon signed-rank test for "is the difference real?"
YAML config files	Human-readable settings	Every emission factor and every port's fuel availability lives in a config file, tagged with its source and confidence — never hard-coded. Change one number, every result updates.
pytest	Automated testing	399 tests, each mapped to an edge case in our test register (e.g. "P-02: fuel must rise with speed"). Judges can be told "every claim on this slide is asserted by a test."
ruff, uv	Code linter and package manager	Keeps code clean and the environment reproducible on any machine
matplotlib	Charts	All 14 charts are generated from the result files by a script, so a chart cannot show a number the code did not produce
<i>Planned:</i> FastAPI, React, Plotly, PostgreSQL, Docker	Backend API, dashboard, database, packaging	For the decision-support screen; Docker so it runs offline on a port's own server (data sovereignty)

The principle behind the choices: every number must be reproducible by anyone who clones the repository and runs four scripts. Nothing depends on a cloud service, a paid licence, or a person's memory. That is what "on-premises, data stays in India" means in practice.

Our results, in plain words

THE NUMBER	WHAT IT MEANS	WHY IT MATTERS
106,922 ship-years, 25,967 ships	How much real, audited data we trained and tested on	Answers "where does your data come from?" better than any simulated dataset could
19.9% MAPE, R² 0.603	On ships never seen in training, our energy prediction is off by about 20% on average and explains 60% of the differences between ships	It is leakage-safe, so the number is real. Per-ship year-to-year variation is ~11% — no method could ever get below that
80.0% coverage	Our "80% confidence band" really does contain the truth 80% of the time	The planner can trust the conservative end of the band
+40% (grey ammonia)	Grey ammonia is 40% worse than diesel over its lifecycle, although a funnel-only tool scores it as perfect	The single clearest reason our tool is needed
1.72% vs 3.1%	Measured methane slip from 745 real LNG ships vs the regulatory default	The two disagree on whether LNG helps at all. Ours is measured
2035	The year an electric tug beats diesel on CO ₂ in India	Honest, specific, and paired with the immediate air-quality win (35.6 t NOx/tug/yr)
30 / 30 exact	On a tiny problem with a provable best answer, our optimizer found it every time	Trust on small problems is the precondition for trust on large ones
p = 0.004	At 60 ships, our optimizer beats NSGA-II and the difference is statistically real	The quantum-inspired advantage shows up where it should — on large problems
–91% for +45%	On a coastal fleet in 2047, the cleanest plan cuts emissions 91% for 45% more cost, with 14 plans between	This is the trade-off a port actually needs to see
399 tests	Automated checks, each tied to an edge case in our register	"Every number on this slide is asserted by a test"

Our honest limitations — say them first

Judges respect a team that knows its own weaknesses. Say these before they are found:

- **Offshore vessels predict badly** (118% error). 96% of them have no design certificate and they only entered the data in 2024. The model correctly flags them as cold-start with wide bands. They need their own model in future.
- **Ro-pax and passenger ships** are weaker (30–34%) because hotel load dominates and utilisation varies wildly.
- **No tug is in the training data.** Tug figures come from physics and published specifications and are labelled as such.
- **Port fuel-availability years are placeholders.** The structure is real; the exact years must be confirmed with each port authority. That is the natural first ask of the Mormugao pilot.
- **The Pareto front thins above ~50 ships** at our test budget. More computing budget fixes it.
- **Nothing has run on quantum hardware.** "Quantum-ready" means the problem is written in the right form and verified.
- **The 12-ship benchmark is a tie**, not a win. Say "tie".

How to present it

Suggested roles (one owner per topic — the test register recommends this)

Owner 1 — Maritime & policy

The problem, India's programmes (GTTP, Harit Sagar, Mormugao, Harit Shrey), the well-to-wake argument, the electrification story, the competitor comparison. Must be able to explain *grey vs green* and *methane slip* from memory.

Owner 2 — Data & prediction

EU MRV, the circularity trap, physics + ML, leakage-safe testing, MAPE / R^2 , cold-start, P10/P90. Must be able to say the accuracy numbers and what split they come from.

Owner 3 — Quantum-inspired optimizer

Q-bits, rotation gates, QPSO, Pareto front, constraints, benchmarking, QUBO. Must be able to say "it is not a quantum computer" clearly, explain the encoding in one diagram, and quote the p-values including the tie.

The three-sentence pitch (memorise it)

"Ports under GTTP and Harit Sagar must decide which ships, which fuels and which speeds — a decision no existing tool makes. We built a planner trained on 106,000 real audited ship-years that counts the whole lifecycle of each fuel — which flips the verdict on three fuels — and a quantum-inspired optimizer, verified against exact solutions, that returns the full cost-versus-emissions trade-off under real Indian port constraints. Our honest finding for India: electrify tugs now for air quality; the carbon win arrives with the grid around 2035."

If asked something you don't know

Say: "That is in our findings document; I will confirm the exact figure rather than guess." Then name the owner who has it. Never invent a number — an invented number that gets caught destroys everything else.

Glossary, A–Z

Ammonia (NH₃) — a fuel with no carbon, so no CO₂ when burned; but its *production* can be very dirty (grey) or clean (green), and burning it produces N₂O.

AR5 / AR6 — the 5th and 6th IPCC assessment reports; they give slightly different GWP multipliers. The EU uses AR5; climate science uses AR6. We support both and say which.

Bunkering — refuelling a ship.

CEA — Central Electricity Authority of India; publishes the grid's CO₂ per unit of electricity.

CH₄ — methane, the main component of natural gas and LNG; ~28–30× CO₂'s warming effect.

CO₂e — CO₂-equivalent; all greenhouse gases converted to one unit using GWP.

Cold-start — a ship the model has almost no information about; it gets a physics estimate and a wide uncertainty band.

Constraint — a rule a plan must obey (deliver all cargo, stay under the emission cap, only use available fuel...).

CP-SAT / MILP — exact mathematical solvers that *prove* the best answer; we use them to verify our method on small problems.

Cube law — a ship's propulsion power rises with speed³; slow down 14→12 knots and power drops ~37%.

Discrete vs continuous — "pick one from a list" (which fuel) vs "any value in a range" (speed).

EMSA — European Maritime Safety Agency; publishes the MRV data.

GT (gross tonnage) — a measure of ship size. MRV covers ships ≥5,000 GT; tugs are ~200–500 GT.

GTTP — Green Tug Transition Programme; India's scheme to replace diesel tugs with electric / clean-fuel ones.

GWP — Global Warming Potential; the multiplier that turns a gas into CO₂e over a stated horizon (we use 100 years).

Grey / blue / green — how a fuel was made: from fossil gas with no capture (grey), with carbon capture (blue), or from renewable electricity (green).

Harit Sagar — India's green-port guidelines: –30% emissions per tonne of cargo by 2030, –70% by 2047.

Harit Shrey — Mormugao Port's incentive scheme that discounts port charges for cleaner ships.

Hotel load — a ship's power use that does not depend on speed (lights, cooling, pumps); large on cruise ships.

Hypervolume — how much of the trade-off space a Pareto front covers; the standard score for comparing multi-objective optimizers.

Knot — 1 nautical mile per hour $\approx$ 1.85 km/h.

Leakage — when test data secretly overlaps training data, making accuracy look better than it is. We split by whole ships to prevent it.

LHV — lower heating value; energy per kilogram of a fuel. Methanol's is about half of diesel's.

LNG — liquefied natural gas; cleaner on CO₂ but leaks methane.

MAPE — mean absolute percentage error; average % by which predictions miss. Lower is better.

Methane slip — unburnt methane escaping from an LNG engine. Default 3.1%; measured 1.72%.

Monotone constraint — telling the machine-learning model an input may only push the answer one way (e.g. a less-efficient design never predicts less energy).

MRV — Monitoring, Reporting and Verification; the EU ship-emissions law and its data.

N₂O — nitrous oxide; $\sim 265\text{--}273\times$ CO₂'s warming effect; formed when ammonia burns.

Nautical mile (nmi) — 1,852 m.

NO_x / SO_x / PM — local air pollutants (nitrogen oxides, sulphur oxides, particulate matter). Harm health; kept separate from CO₂e.

NSGA-II — the standard genetic algorithm for multi-objective problems; our main baseline.

P10 / P50 / P90 — low / middle / high estimates; the P10–P90 band should contain the truth 80% of the time.

p-value — probability a result is due to luck; below 0.05 counts as significant.

Pareto front — the set of plans where nothing can improve without something else worsening; the output of the optimizer.

Pilot fuel — the small dose of diesel a methanol or ammonia engine needs to ignite; we count it.

Q-bit — a decision stored as a probability (amplitudes), not a fixed yes/no; the core of the quantum-inspired encoding.

QIEA — Quantum-Inspired Evolutionary Algorithm; Q-bits plus rotation gates. Used for the discrete decisions.

QPSO — Quantum-behaved Particle Swarm Optimization; a velocity-free swarm search. Used for continuous speeds.

QUBO — Quadratic Unconstrained Binary Optimization; the form quantum annealers consume. Our "quantum-ready" claim.

R² — share of variation the model explains, 0 to 1.

Rotation gate — the update that nudges a Q-bit's probability toward the best solution; the "quantum update mechanism".

Seed — the starting number for a random process; the same seed gives the same result (reproducibility). We use 30 different seeds for benchmarks.

Shaft energy — useful work delivered at the propeller shaft; the fair basis for comparing fuels with batteries.

SHA-256 — a digital fingerprint of a file; proves which exact data version produced a result.

Shore power / OPS / cold ironing — plugging a docked ship into the grid.

Slow steaming — deliberately sailing slower to save fuel (cube law).

T&D losses — electricity lost in wires between power station and ship (~17% in India).

THETIS-MRV — EMSA's online system that publishes the MRV data.

Well-to-wake — the full lifecycle of a fuel: making it (well-to-tank) plus burning it (tank-to-wake).

Wilcoxon signed-rank test — a statistical test for whether two paired sets of results genuinely differ.

XGBoost — the machine-learning method we use for the ship-specific efficiency level.

Companion documents: [docs/REPORT_FOR_PPT.pdf](#) (the results, slide by slide) · [docs/FINDINGS_PHASE1.md](#) , [FINDINGS_PHASE2.md](#) , [FINDINGS_PHASE3.md](#) (the full technical record) · [docs/TEST_AND_EDGE_CASES.md](#) (the test register every ID refers to).